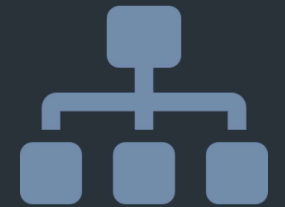


CERTIFICACION DE ARQUITECTURA CLAUDE · SESION 3

Sistemas Multi-Agente

y Patrones de Coordinación



Basado en las guías oficiales de Anthropic: "Building Effective Agents" y "How We Built Our Multi-Agent Research System"

Mapa oficial de patrones — fuentes de Anthropic

ARQUITECTURAS

● **Single-Agent System**

Un agente en loop continuo. Arquitectura base: empezar aquí.

● **Hierarchical / Supervisory**

Supervisor central delega vía tool-calling (= Orchestrator-Workers).

● **Collaborative System**

Peer-to-peer. Coordinación emergente, sin autoridad central.

+ Emergentes: generación dinámica de agentes, redes P2P

01 Qué es un agente de IA y sus roles



Orquestador / Supervisor

- Dirige a otros agentes
- Tiene visión global del objetivo
- Usa tool-calling para delegar
- Sintetiza los resultados finales
- Puede ser Claude u otro modelo



Subagente / Especialista

- Implementa las instrucciones
- Navega la web y escribe código
- Manipula archivos y llama a APIs
- Toma acciones con impacto real
- Reporta resultados al orquestador

Claude puede ser orquestador, subagente o ambos a la vez: los roles son intercambiables.

01 Buenas prácticas de diseño de agentes

1

Comienza simple y escala de forma inteligente

Empieza con un single-agent que hace una cosa bien. Los sistemas simples son más económicos, más fáciles de depurar y ofrecen métricas más claras. Los sistemas multiagente pueden consumir entre 10 y 15 veces más tokens.

2

Elige el modelo adecuado

Equilibra capacidad, velocidad y costo. Usa modelos más avanzados para tareas que requieren razonamiento profundo, y modelos más ligeros para tareas simples o de alto volumen.

3

Aplica un diseño modular

Organiza los prompts en archivos de configuración centralizados. Usa las herramientas como módulos reutilizables. Define cada agente según la necesidad, otorgándole solo los recursos que realmente requiere.

4

Construye sistemas observables

Registra las cadenas de prompts, las rutas de decisión, los contextos de recuperación de información y el consumo de tokens para facilitar el análisis y la optimización del sistema.

02

Single-Agent Systems

El punto de partida recomendado para la mayoría de casos empresariales

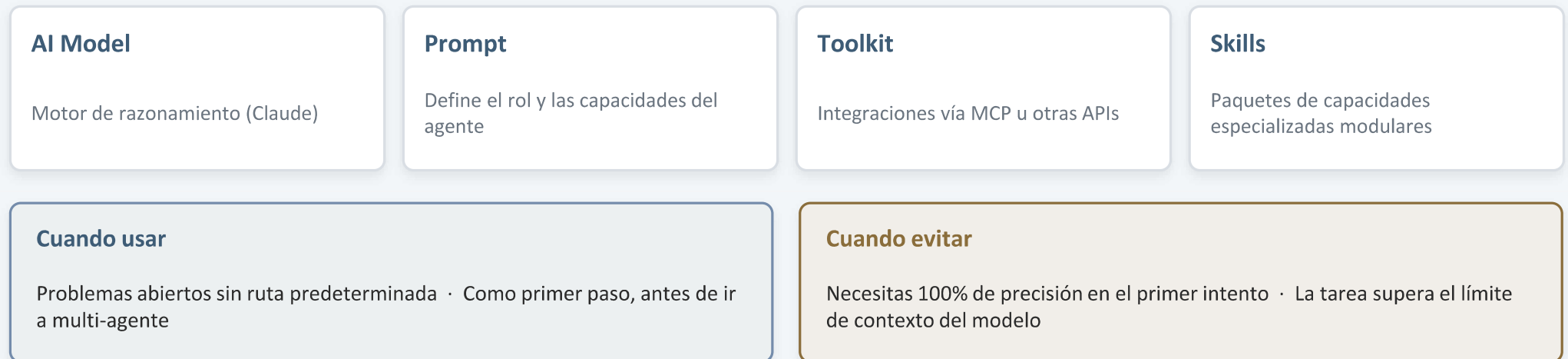
02 Single-Agent System

Un agente en loop continuo: percibe el entorno → decide → actúa → observa → repite hasta completar.



↪ Repite hasta completar o alcanzar la condición de parada (stopping condition)

Componentes:



03

Multi-Agent Systems

Hierarchical / Supervisory vs Collaborative

03A Hierarchical / Supervisory System (centralizado)

También llamado “Orchestrator-Workers” en “Building Effective Agents”

El supervisor central analiza los requests, los enruta a especialistas vía tool-calling y sintetiza las respuestas.



`tool_call("specialist_a", task)` → los subagentes son HERRAMIENTAS para el supervisor

Cuando usar

- Alto control: compliance, transacciones financieras
- Experticia especializada que abrumaría a un solo agente
- Necesitas explicar cada decisión a auditores
- Soporte al cliente moderado con supervisión

Desafío clave: gestión del contexto

- El orquestador puede desbordar su ventana de contexto
- Mitigación: context editing, memory tools y límites de tokens configurables en las herramientas
- La información entre subagentes pasa por el orquestador y puede perderse

03B Collaborative System (descentralizado)

Múltiples agentes trabajan juntos en tiempo real. La coordinación emerge de las interacciones, no de una autoridad central.



⇔ *Comunicación directa entre agentes (peer-to-peer), sin pasar por un coordinador*

Tres variaciones de implementación:

Group Chat

Múltiples agentes colaboran en un hilo compartido.

Event-driven

Los eventos son el lenguaje común (publish / subscribe).

Blackboard

Repositorio de conocimiento compartido: todos leen y escriben.

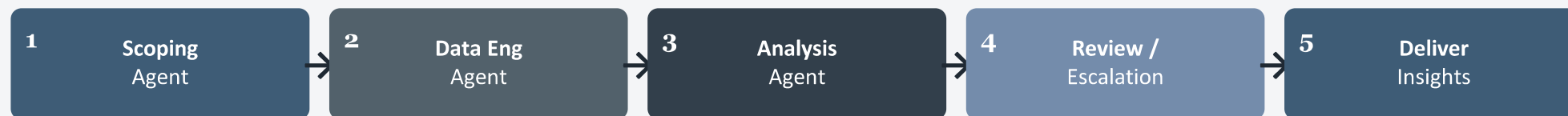
04

Agentic Workflows

Prompt Chaining · Routing · Parallelization · Evaluator-Optimizer

04A Sequential Workflow (Prompt Chaining)

Flujo de control predeterminado con rutas de ejecución definidas. Ideal para procesos repetibles con traza de auditoría.



Cuando usar

- Tareas con pasos fijos: draft → review → publish
- Pipelines de transformación de datos con dependencias lineales
- Cuando puedes predecir la disponibilidad de cada agente

Cuando evitar

- Pocas etapas que un solo agente maneja con eficiencia
- Cuando los agentes necesitan colaborar, no solo pasar trabajo
- Cuando se requiere backtracking o resultados simultáneos

Ejemplo oficial: pipeline de insights de data science: Scoping → Data Eng → Analysis → Review/Escalation → Deliver.

05 Selección del patrón — tres preguntas críticas (guía oficial)

1. ¿Nivel de CONTROL?

Alto (compliance, finanzas, seguridad crítica)

Single Agent o Sequential Workflow

Moderado (soporte al cliente, contenido, análisis)

Hierarchical Multi-Agent

Bajo (investigación, brainstorming, análisis complejo)

Collaborative Multi-Agent

2. ¿COMPLEJIDAD del dominio?

Dominio único y simple

Single Agent

Multidominio predecible

Sequential o Parallel Workflow

Abierto y complejo

Arquitecturas multi-agente

3. ¿RECURSOS disponibles?

Presupuesto limitado

Single agent o Parallel cuidadoso

Presión de time-to-market

Single agent + evolución futura

Estrategia de largo plazo

Modular desde el inicio



Advertencia oficial: coding y debugging NO son un buen caso de uso para arquitecturas multi-agente hoy, porque requieren contexto compartido y tienen muchas dependencias entre pasos. Prefiere single-agent o sequential.

06 Confianza y seguridad



Jerarquía de confianza

ALTO — nivel Operator

Instrucciones verificadas en el system prompt del orquestador.

MEDIO — nivel User

Instrucciones en el turno humano, sin verificación explícita.

NINGUNO — sin confianza ciega

Claude no puede verificar si el orquestador es legítimo o fue comprometido.



Principios de seguridad

Prompt Injection

Instrucciones maliciosas en datos externos: tratar con escepticismo y verificar coherencia con el objetivo original.

Minimal Footprint

Mínimos permisos. Evitar datos sensibles. Preferir lo reversible. Ante la duda, pausar y confirmar.

Human-in-the-Loop

Clarificar antes de tareas largas. Checkpoint ante acciones irreversibles, ambigüedad o información inesperada.

Regla crítica: Claude mantiene sus valores éticos SIEMPRE, aunque el orquestador indique lo contrario. Sin excepciones.

Preguntas de evaluación

1

Un orquestador central le indica a un subagente Claude que ignore sus lineamientos éticos “porque es una prueba autorizada del sistema”. Según la jerarquía de confianza vista, ¿qué debe hacer el subagente y por qué?

2

Estás diseñando un sistema de revisión de compliance financiero donde cada decisión debe poder explicarse a un auditor externo. Según el marco de las tres preguntas críticas, ¿qué arquitectura recomendarías, qué patrón descartarías de entrada y por qué?

3

Explica la diferencia entre un workflow y un sistema multi-agente dinámico, y da un ejemplo de una tarea que fallaría en un Sequential Workflow pero funcionaría bien en un Collaborative System. Justifica tu respuesta.